

Chapter 5: Pointers and File Handling

अध्याय 5: प्वाइंटर एवं फाइल हैंडलिंग

UP Board Class 10 – Computer (836) | Papers: 2023 (05000/226), 2024 (40000/4031), 2025 (L/1076), 2026 (L/S-1027, 836 UA)
2022 paper not uploaded, so not included / 2022 का पेपर अपलोड नहीं था

Part 1: Pointers / प्वाइंटर

Q1 | 2026 | Set: L/S-1027 (836 UA) | Part-A MCQ 6 | 1 Mark

हि: कौन-सा प्वाइंटर प्रकार किसी भी डेटाटाइप का पता संग्रहित कर सकता है? (A) int (B) void (C) char (D) float प्वाइंटर

En: Which pointer type can store the address of any datatype? (A) int pointer (B) void pointer (C) char pointer (D) float pointer

Answer: (B) void pointer

हि: void प्वाइंटर जेनेरिक प्वाइंटर है: void *p; इसमें किसी भी टाइप का पता रखा जा सकता है। (dereference करने से पहले उसे सही टाइप में cast करना पड़ता है।) बाकी प्वाइंटर केवल अपने टाइप का पता रखते हैं।

En: A void pointer is a generic pointer: void *p; It can hold the address of any type (cast it to the right type before dereferencing). Other pointers hold addresses of their own type only.

Q2 | 2026 | Set: L/S-1027 (836 UA) | Part-A MCQ 12 | 1 Mark

हि: आपके पास arr[5] ऐसे के पहले तत्व का प्वाइंटर 'ptr' है। तीसरे तत्व तक पहुँचने के लिए आप लिख सकते हैं: (A) *(ptr+3) (B) *(ptr+2) (C) *(ptr+1) (D) *(ptr)

En: You have a pointer 'ptr' to the first element of arr[5]. To access the third element, you can write:

Answer: (B) *(ptr + 2)

हि: इंडेक्स 0 से शुरू होता है: पहला = *(ptr+0), दूसरा = *(ptr+1), तीसरा = *(ptr+2)। यह arr[2] के बराबर है।

En: Indexing starts at 0: first = *(ptr+0), second = *(ptr+1), third = *(ptr+2), which equals arr[2].

Q3 | 2026 | Set: L/S-1027 (836 UA) | Part-A MCQ 13 | 1 Mark

हि: यदि एक प्वाइंटर p किसी पूर्णांक चर x की ओर इशारा करता है, तो अभिव्यक्ति *p किसे संदर्भित करती है? (A) x का पता (B) x का मान (C) एक स्थिर मान (D) एक अपरिभाषित मान

En: If a pointer p points to an integer variable x, the expression *p refers to: (A) address of x (B) value of x (C) constant value (D) undefined value

Answer: (B) The value of x / x का मान

हि: * (dereference ऑपरेटर) प्वाइंटर में रखे पते पर रखा मान देता है। x का पता p है (p = &x), और मान *p है।

En: The * (dereference) operator gives the value stored at the address held by the pointer. The address of x is p (p = &x) and its value is *p.

Q4 | 2025 | Set: L/1076 | Part-A MCQ 9 | 1 Mark

हि: किस ऑपरेटर का उपयोग प्वाइंटर को दर्शाने के लिए किया जाता है? (A) & (B) * (C) % (D) @

En: Which operator is used to denote a pointer? (A) & (B) * (C) % (D) @

Answer: (B) *

हि: प्वाइंटर घोषित करने और उसका मान पाने में * लगता है (int *p;)। & "address-of" ऑपरेटर है जो चर का पता देता है।

En: * is used to declare a pointer and to access its value (int *p;). & is the "address-of" operator that gives a variable's address.

Q5 | 2025 (Set L/1076, Part-B VSA Q1) and 2024 (Set 836 40000/4031, Part-B VSA Q6) | 1 Mark each

हि: 'C' में प्वाइंटर क्या है? / प्वाइंटर किसे कहते हैं?

En: What is a pointer in 'C'? / What is pointer?

Answer / उत्तर

हि: प्वाइंटर एक विशेष वैरिएबल है जो किसी दूसरे वैरिएबल का मेमोरी पता (address) संग्रहित करता है। घोषणा: data_type *name;

En: A pointer is a special variable that stores the memory address of another variable. Declaration: data_type *name;

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *p = &x;          // p stores address of x
    printf("%d\n", x);     // 10
    printf("%d\n", *p);    // 10 (value at address p)
    *p = 25;               // changes x
    printf("%d", x);       // 25
    return 0;
}
```

Uses / उपयोग: dynamic memory, arrays, call by reference, data structures / डायनामिक मेमोरी, ऐरे, कॉल बाय रेफरेंस।

Part 2: File Handling / फाइल हैंडलिंग

Q6 | 2024 | Set: 836 (40000/4031) | Part-A MCQ 11 | 1 Mark

हि: C फाइल प्वाइंटर घोषित करने के लिए उपयोग किया जाने वाला कीवर्ड क्या है ? (A) file (B) FILE (C) FILE FP (D) filefo

En: What is the keyword used to declare a C file pointer?

Answer: (B) FILE

हि: फाइल प्वाइंटर इस प्रकार घोषित होता है: FILE *fp; (FILE बड़े अक्षरों में लिखना जरूरी है; C केस-सेंसिटिव है।)

En: A file pointer is declared as FILE *fp; (FILE must be in capital letters; C is case-sensitive.)

Q7 | 2024 | Set: 836 (40000/4031) | Part-A MCQ 12 | 1 Mark

हि: C फाइल डेटा का प्रकार क्या है ? (A) केवल स्ट्रक्चर (B) केवल यूनियन (C) केवल यूजर डिफाइन्ड डाटा टाइप (D) इनमें से कोई नहीं

En: What is a C FILE data type? (A) Structure only (B) Union only (C) User defined data type only (D) None of these

Answer: (D) None of these (see note) / इनमें से कोई नहीं

हि: FILE, stdio.h में typedef द्वारा बनाया गया स्ट्रक्चर टाइप है, यानी यह स्ट्रक्चर भी है और यूजर-डिफाइन्ड टाइप भी। विकल्पों में "केवल" शब्द है, इसलिए तर्क से (D) सही बैठता है। नोट: कुछ उत्तर-कुंजियाँ (A) "Structure" भी देती हैं, इसलिए अपनी पुस्तक/शिक्षक से मिलान कर लें।

En: FILE is a structure type created with typedef in stdio.h, so it is both a structure and a user-defined type. Since the options say "only", (D) fits logically. Note: some answer keys give (A) Structure, so cross-check with your textbook/teacher.

Q8 | 2026 | Set: L/S-1027 (836 UA) | Part-A MCQ 20 | 1 Mark

हि: फाइल हैंडलिंग में, बिना मौजूदा डेटा को ओवरराइट किए कौन-सा मोड फाइल को पढ़ने और लिखने के लिए खोलता है ? (A) "w" (B) "w+" (C) "a+" (D) "r+"

En: Which mode opens a file for reading and writing without overwriting existing data?

Answer: (C) "a+"

हि: "a+" फाइल को पढ़ने और जोड़ने (append) के लिए खोलता है; नया डेटा अंत में जुड़ता है, पुराना डेटा सुरक्षित रहता है। "w" और "w+" पुरानी फाइल का डेटा मिटा देते हैं।

En: "a+" opens the file for reading and appending; new data is added at the end and old data is kept. "w" and "w+" erase the old contents.

Mode	Meaning / अर्थ
r	Read existing file / मौजूदा फाइल पढ़ना
w	Write; creates or erases file / लिखना; नई बनाता या पुरानी मिटाता है
a	Append at end / अंत में जोड़ना
r+	Read and write, file must exist / पढ़ना व लिखना, फाइल मौजूद होनी चाहिए
w+	Read and write; erases old data / पढ़ना व लिखना; पुराना डेटा मिटाता है
a+	Read and append / पढ़ना व जोड़ना

Q9 | 2024 | Set: 836 (40000/4031) | Part-B Short Q11(घ / d) | 2 Marks

हि: फाइल हैंडलिंग से आप क्या समझते हैं ?

En: What do you mean by file handling?

Answer / उत्तर

हि: फाइल हैंडलिंग का अर्थ है C प्रोग्राम द्वारा डिस्क पर रखी फाइलों को बनाना, खोलना, पढ़ना, लिखना और बंद करना। इससे डेटा प्रोग्राम बंद होने के बाद भी स्थायी रूप से सुरक्षित रहता है। इसके लिए FILE *fp और fopen(), fclose(), fprintf(), fscanf() जैसे फंक्शन प्रयोग होते हैं।

En: File handling means creating, opening, reading, writing and closing files stored on disk from a C program. It keeps data permanently even after the program ends. It uses FILE *fp and functions such as fopen(), fclose(), fprintf(), fscanf().

Q10 | 2025 | Set: L/1076 | Part-B Short Q11(ख / b) | 2 Marks

हि: 'सी' में फाइल हैंडलिंग क्या है ? 'सी' में कोई दो फाइल हैंडलिंग फंक्शन लिखें।

En: What is file handling in 'C'? Write any two file handling functions in 'C'.

Answer / उत्तर

हि: फाइल हैंडलिंग = प्रोग्राम से फाइल को बनाना, खोलना, पढ़ना, लिखना और बंद करना (डेटा को स्थायी रूप से रखने के लिए)।

En: File handling = creating, opening, reading, writing and closing files from a program, to store data permanently.

Function	Use / उपयोग
fopen()	Opens/creates a file: fp = fopen("a.txt", "w"); / फाइल खोलता या बनाता है
fclose()	Closes the file: fclose(fp); / फाइल बंद करता है

Other / अन्य: fprintf(), fscanf(), fgetc(), fputc(), fgets(), fputs(), fread(), fwrite(), feof().

हि: C में फाइल बनाने और खोलने की प्रक्रिया को उदाहरण फंक्शन सहित समझाइए।

En: Explain the process of creating and opening a file in C with example functions.

Answer / उत्तर

Steps / चरण:

1. फाइल प्वाइंटर घोषित करें / Declare a file pointer: `FILE *fp;`
2. `fopen("filename", "mode")` से फाइल खोलें या बनाएँ / Open or create using `fopen()` — "w" नई फाइल बनाता है, "r" मौजूदा फाइल खोलता है।
3. जाँचें कि `fp` NULL तो नहीं (फाइल न खुलने पर NULL मिलता है) / Check `fp` for NULL.
4. `fprintf()/fscanf()` से लिखें-पढ़ें / Write or read.
5. `fclose(fp)`; से बंद करें / Close the file.

```
#include <stdio.h>
int main()
{
    FILE *fp;
    char ch;

    fp = fopen("student.txt", "w");    // create file
    if (fp == NULL) { printf("Cannot open file"); return 1; }
    fprintf(fp, "Hello UP Board");    // write
    fclose(fp);

    fp = fopen("student.txt", "r");    // open existing file
    while ((ch = fgetc(fp)) != EOF)    // read char by char
        printf("%c", ch);
    fclose(fp);
    return 0;
}
// Output: Hello UP Board
```